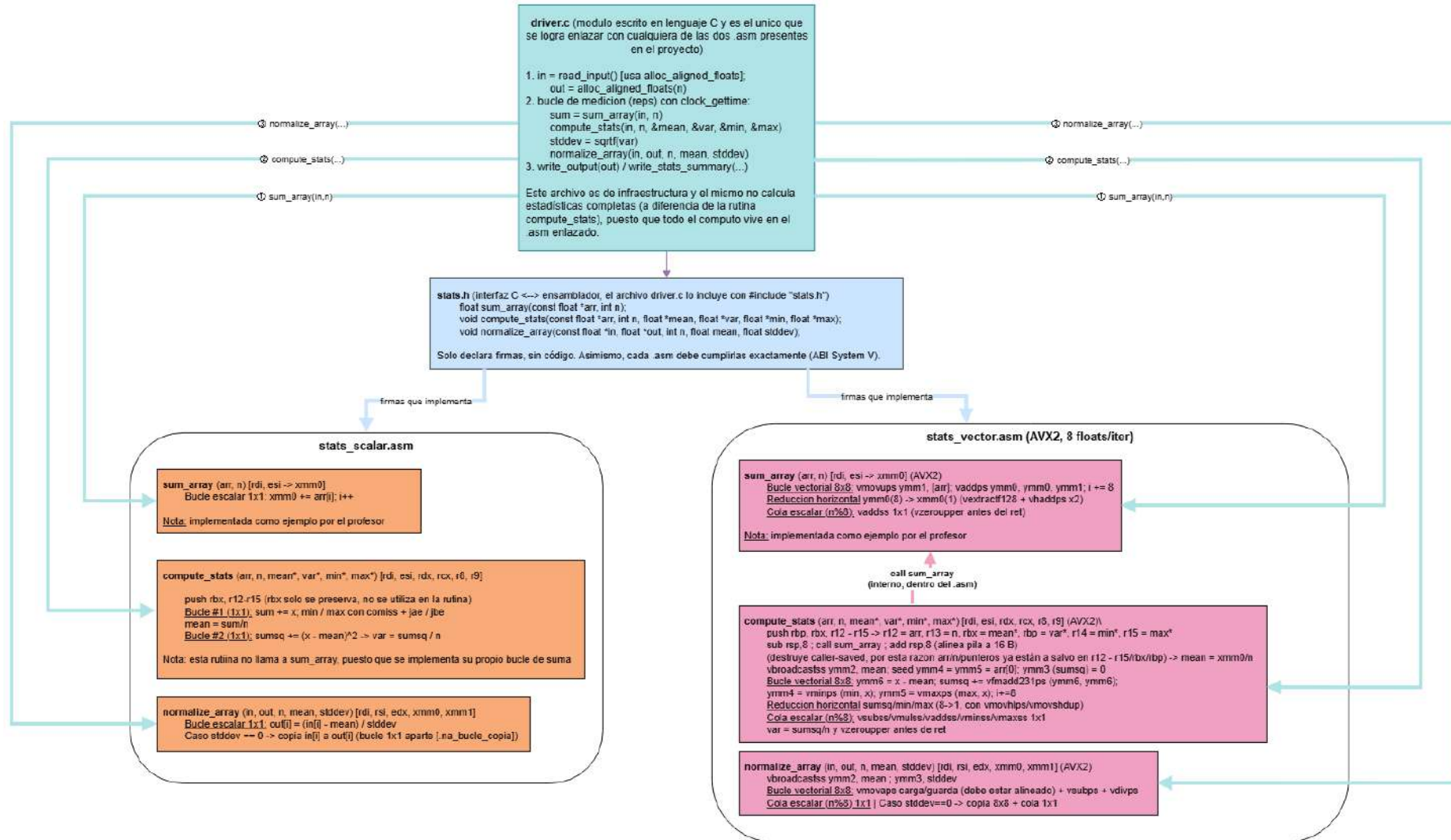


Diagrama de bloques: arquitectura de software (version Escalar vs Vectorial [AVX2])

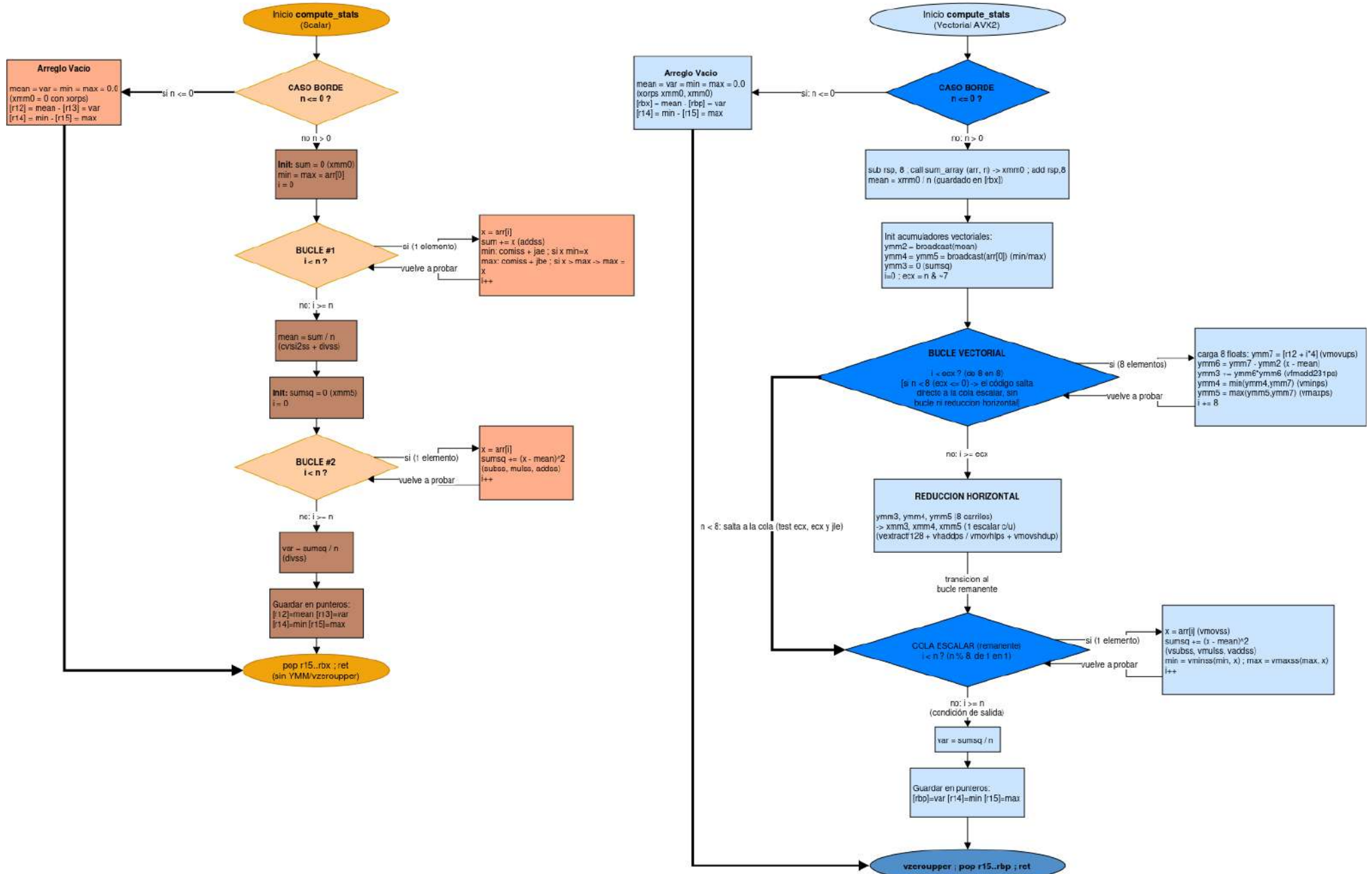


Nota: se compilan los dos binarios distintos a partir del mismo archivo `driver.c`, para de esta manera poder enlazar con `stats_scalar.asm` uno y otro con `stats_vector.asm`, puesto que ambos exponen exactamente las mismas 3 funciones (misma firma - ABI). Asimismo, estas firmas se declaran en `stats.h`.

Convención compartida por ambas versiones: System V ABI

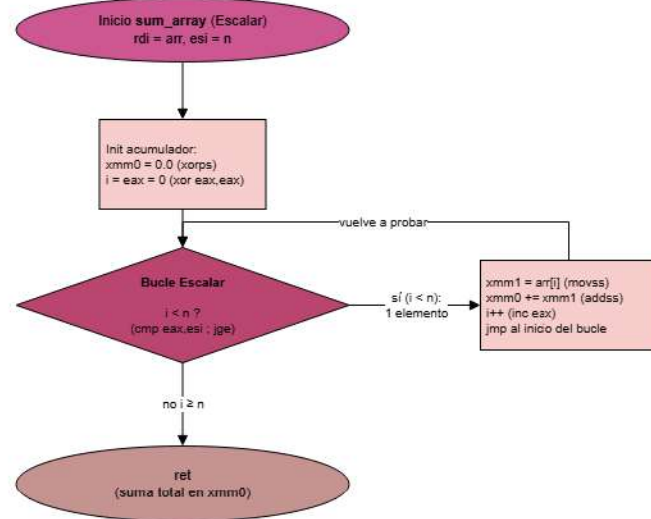
- Enteros / punteros (orden de argumentos): `rdi, rsi, rdx, rcx, r8, r9`
- Flotantes (orden de argumentos): `xmm0, xmm1, xmm2, ...`
- Valor de retorno de tipo `float`: `xmm0`
- Registros callee-saved (los de tipo callee se deben preservar si se utilizan): `rbx, rbp, r12-r15`
- Registros callee-saved (si se pueden destruir): `rax, rcx, rdx, rsi, rdi, r8-r11`

Flujo de control del bucle principal de compute_stats: Escalar (1 elem/iter) vs Vectorial AVX2 (8 elem/iter + cola)

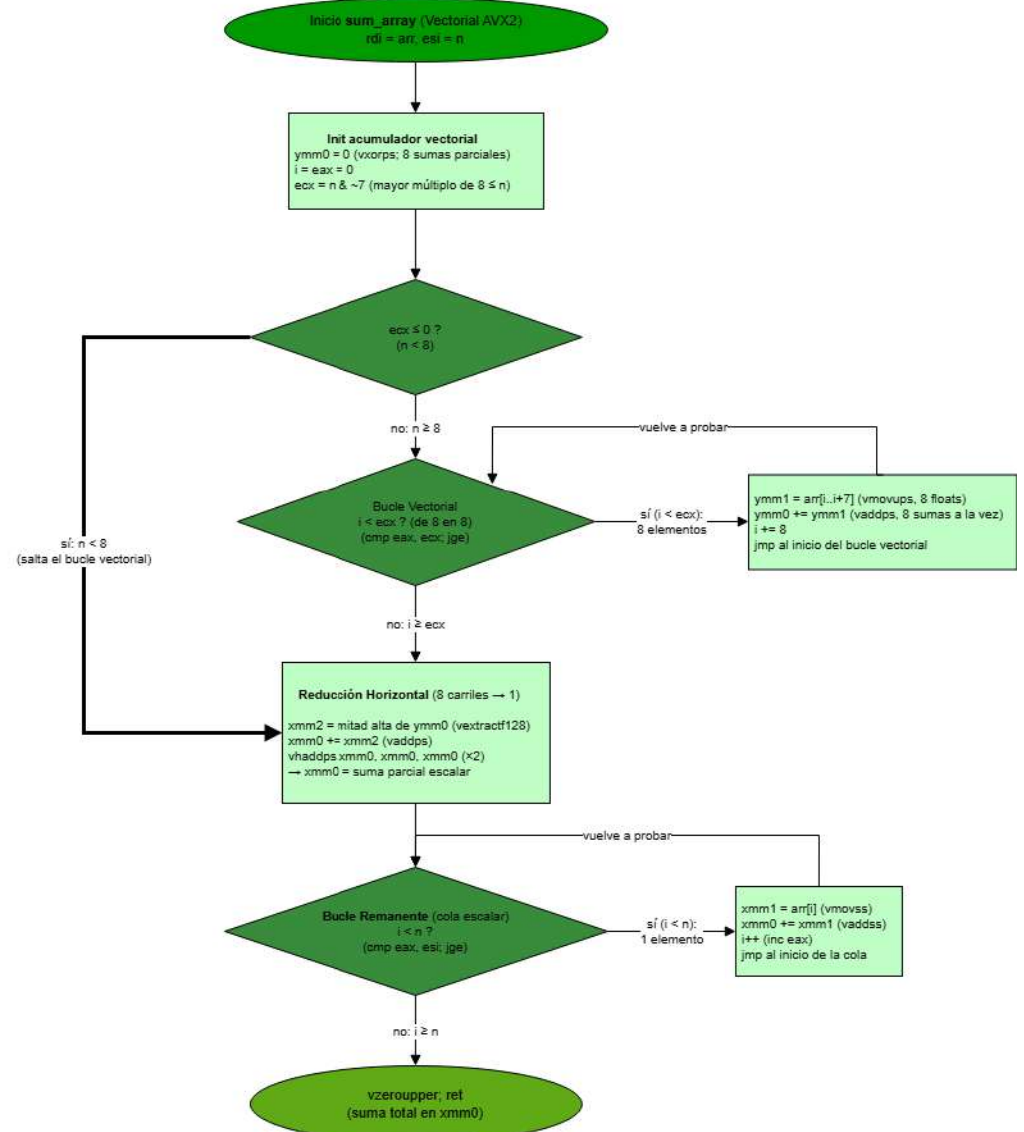


Flujo de control de sum_array: Escalar (1 elem/iter) vs Vectorial AVX2 (8 elem/iter + cola)

Escalar (stats_scalar.asm) — 1 elemento por iteración y sin ningún remanente

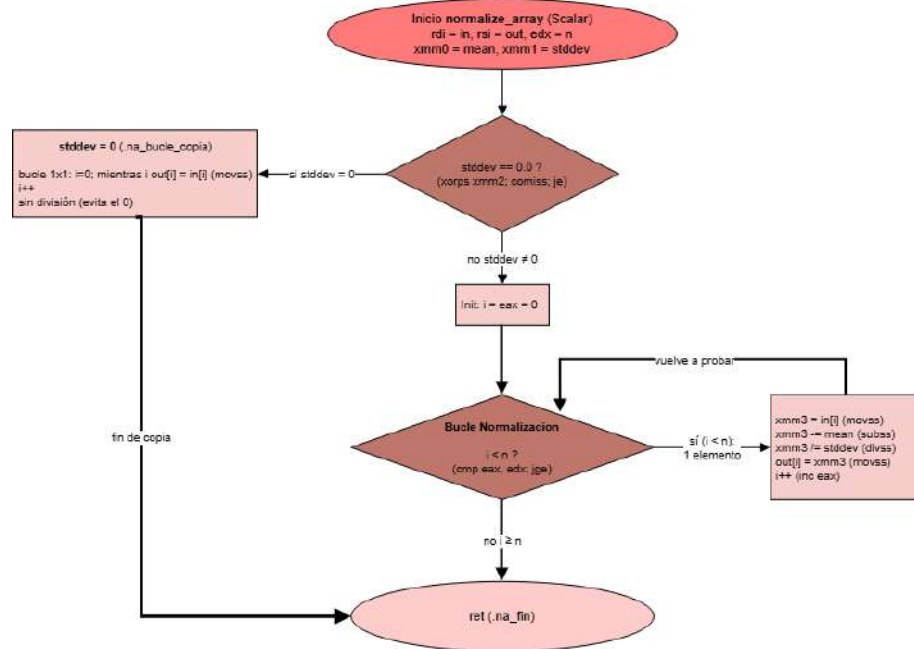


Vectorial AVX2 (stats_vector.asm) — 8 elementos por iteración + cola escalar



Flujo de control de normalize_array: Escalar (1 elem/iter) vs Vectorial AVX2 (8 elem/iter + cola)

Escalar (stats_scalar.asm) — 1 elemento por iteración, sin ningún remanente



Vectorial AVX2 (stats_vector.asm) — 8 elementos por iteración + cola escalar

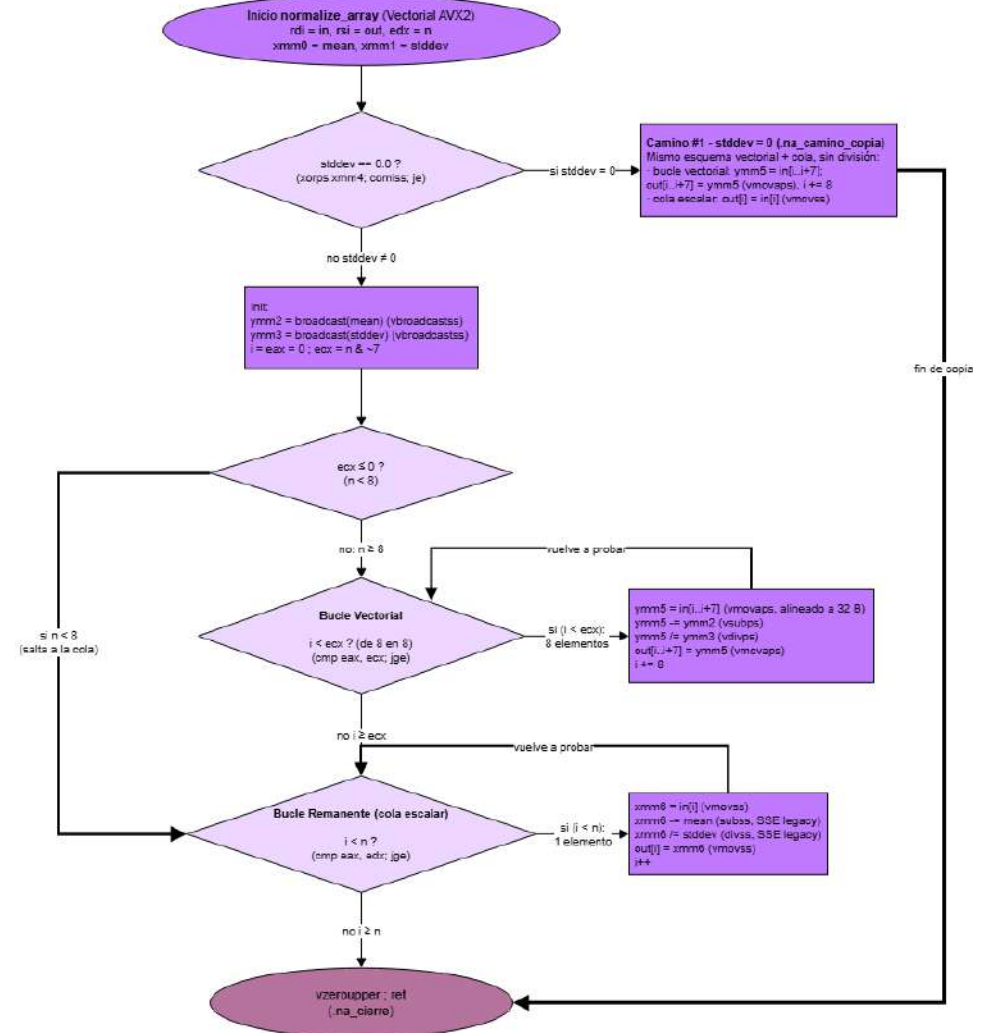


Diagrama de memoria: disposicion de in/out en normalize_array, alineacion a 32 bytes y avance del puntero (escalar vs vectorial)

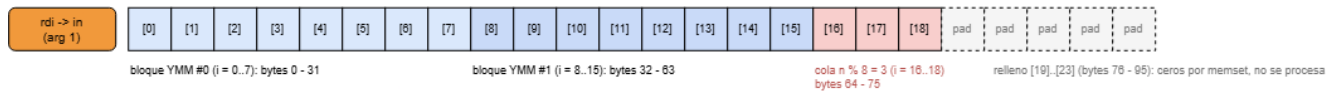
driver.c: alloc_aligned_floats(count) -> aligned_alloc(VEC_ALIGN=32, padded), luego en memset(p, 0, padded)
bytes = count*4 y padded = ((bytes + 31)/32)*32. Asimismo si padded == 0 -> padded = 32 (aligned_alloc exige un tamaño múltiplo de la alineación; la base queda alineada por VEC_ALIGN=32)
Se invoca con count = max(n,1); incluso n = 0 reserva 32 B (nunca NULL).

Un ejemplo puntual para este diagrama:

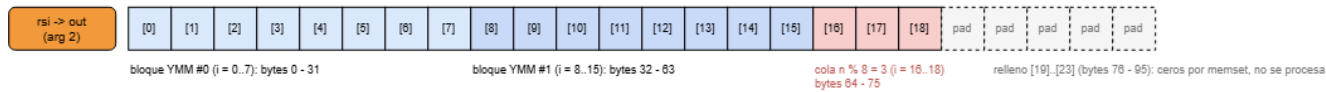
n = 19 -> 76 B -> padded = 96 B (24 floats). Las posiciones [19]..[23] quedan en 0.0 por el memset y nunca se procesan.

Dos arreglos independientes con este mismo mecanismo: 'in' (dentro de read_input(), donde luego fread carga los n floats) y 'out' (en main, salida de normalize_array).

Arreglo 'in' (float in[n]) - direccion base multiplo de 32



Arreglo 'out' (float out[n]) - direccion base multiplo de 32, reserva independiente



Formula de direccionamiento: elemento[i] = base + rax*4 (cada float ocupa 4 bytes).

Base = rdi en sum_array, normalize_array (in=rdi, out=rsi) y compute_stats escalar.

Asimismo en compute_stats vectorial la base es r12 (copia del arreglo: el 'call sum_array' destruye rdi). Por otra parte, es importante indicar que rsi solo apunta a 'out' en normalize_array; en sum_array / compute_stats, esi = n (no es puntero).

sum_array / compute_stats (vectorial) cargan con la instrucción vmovups (en este caso no exigen alineación; funcionan aunque el arreglo de datos no este alineado a 32B).

normalize_array (vectorial) usa vmovaps sobre 'in' y 'out' en el bucle de 8 y en el camino stdev == 0, tiene que tener alineación a 32B de forma obligatoria (si no, SIGSEGV).

Finalmente esto es valido porque ambos se reservaron con aligned_alloc(32,...) y rax avanza de 8 en 8 -> cada bloque empieza en una base + múltiplo de 32.

Por ultimo, la cola (n%8) siempre usa la instrucción vmovss (4 B, sin requisito de alineación), por esa razon el relleno del final no se procesa.

Avance del Puntero (Versión Escalar)

rax (i) += 1 por iteracion -> offset avanza +4 bytes/iteracion

Se lee/escribe un dato tipo float por vuelta del bucle en todo el arreglo de datos (sin distinción alguna de cola).

Ejemplo puntual: n=19: 19 iteraciones, offsets 0, 4, 8, ..., 72 (compute_stats hace 2 pasadas de este tipo).

Avance del Puntero (Versión Vectorial -AVX2)

Tramo vectorial: rax (i) += 8 por iteracion -> offset avanza +32 bytes por iteración.

(exactamente el ancho de un registro YMM: 8 floats x 4 bytes)

Tramo de cola (n % 8, resaltado en rojo arriba): rax += 1 -> +4 bytes por iteración, idéntico al modo escalar, para los elementos que no completan un bloque de 8.

Ejemplo puntual: n = 19: 2 iteraciones vectoriales (offsets 0 y 32) + 3 de cola (offsets 64, 68, 72) = 5 iteraciones.